

# Current State: Conversational Copilot Development

*Productivity is improving, but delivery remains user-driven, conversational, and difficult to govern.*

## HOW WORK HAPPENS TODAY

- 1 Developer prompt
- 2 Requirements discussed in chat
- 3 Agent makes design assumptions
- 4 Code generated iteratively
- 5 Re-prompt, patch, and retry
- 6 Review and validation happen later

## VALUE TODAY

-  Faster exploration and prototyping
-  Quicker code and documentation drafts
-  Low barrier to adoption
-  Improved individual developer productivity

## CURRENT LIMITATIONS

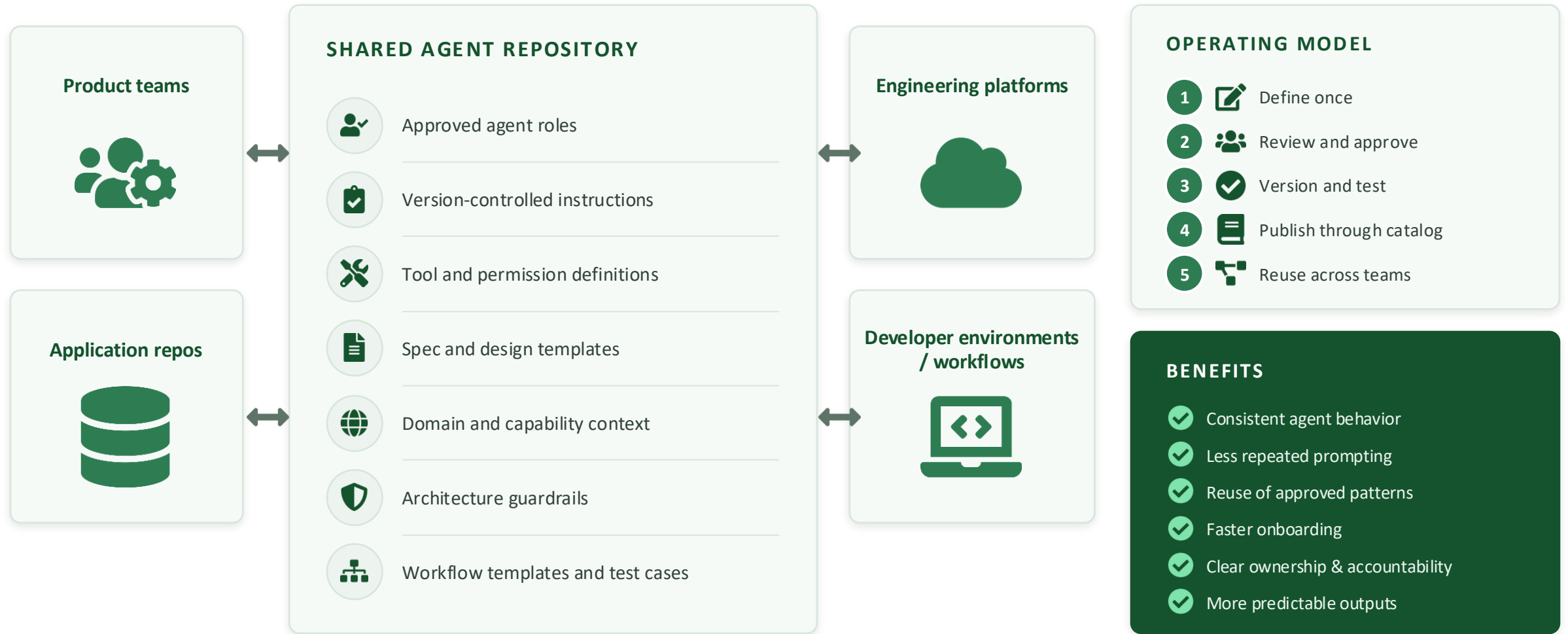
-  Requirements and decisions stay hidden in conversations
-  Architecture decisions can be implicit or inconsistent
-  Context must be repeatedly supplied
-  Agent behavior varies by user and prompt
-  Limited traceability across requirement, design, and code
-  More rework when assumptions are incorrect
-  Governance occurs late in the process



Effective for individual assistance, but **not yet a repeatable enterprise delivery model.**

# Shared Agent Foundation: Approved Agents from a Common Repository

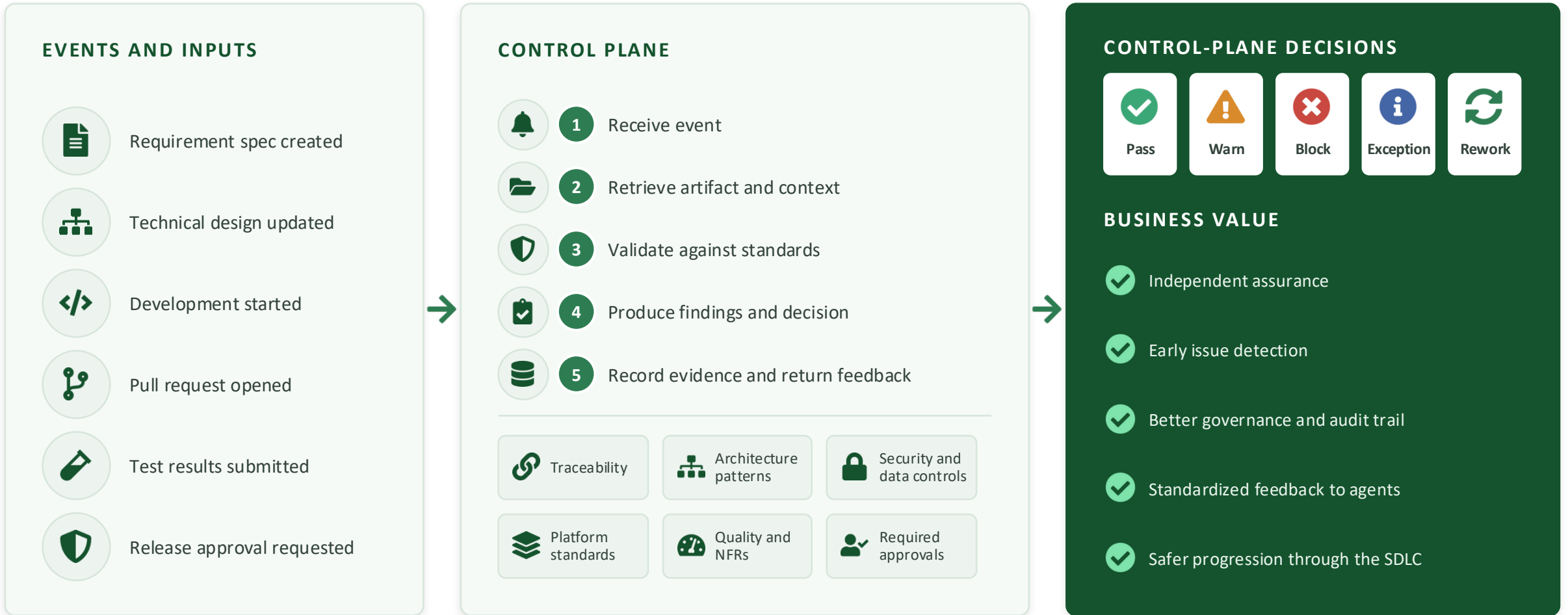
*Move from individual prompting to reusable, version-controlled, enterprise-approved agents.*



Centrally governed, locally executed, and **safely reused**.

# Architecture Control Plane: Event-Driven Assurance

*Independently validate agent-produced artifacts at key lifecycle events as work progresses.*



**Reactive, not mandated.** Validates after the fact and only on emitted events — gaps remain between events and on ungoverned paths. **Preventive, gate-before-execute governance arrives in the Target State.**

# Target State: Governed, Capability-Grounded Agentic SDLC

*Specialized agents collaborate across the lifecycle using approved context, shared controls, and human accountability.*

## GOVERNANCE LAYERS



Shared agent repository



Approved specifications



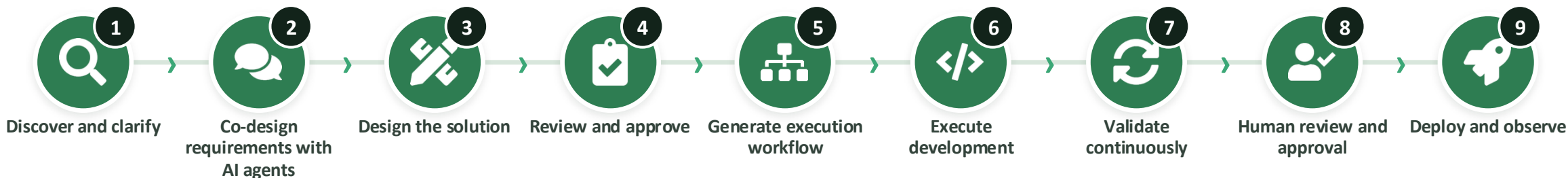
Control plane



Human approvals

## AGENTIC LIFECYCLE

*Co-design = human leads and AI agents shaping requirements and design together — not solo prompting.*



## CAPABILITY-GROUNDED CONTEXT



Business capability & domain vocabulary



Product & customer context



Approved architecture patterns



Platform & technology standards



Security, risk & compliance



Repository & codebase context

## TARGET OUTCOMES



Clear source of truth



Lower unsupported assumptions



Reduced prompting and rework



Repeatable implementation



Built-in traceability



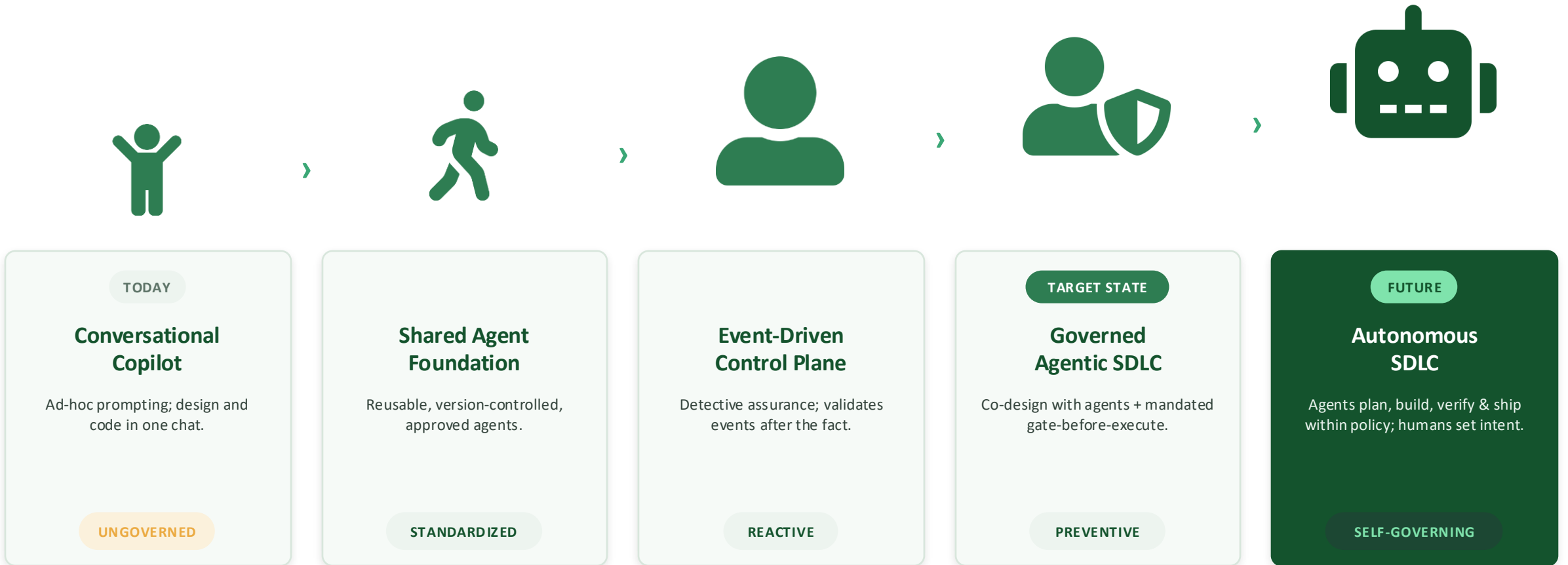
Predictable quality and compliance



Co-design with AI agents on approved context. Execute from an approved specification. **Govern through evidence.**

# The Maturity Curve: From Conversational Copilot to Autonomous SDLC

*Each stage compounds capability and governance — autonomy is the horizon, earned on top of mandated control.*



 Autonomy is the destination — but it is **earned on top of mandated governance**: agents gain independence as verification and guardrails prove out.